

GAME PROGRAMMING PATTERNS

ステートパターン

状態ごとの処理を整理し、見通しの良いコードを書く

対象：3年生

STAT
E

参考資料について

- この授業は、Unity 公式の eBook 「Level up your code with game programming patterns」を参考にしています
- サンプルプロジェクト（GitHub）にも、ステートパターンの実装例が収録されています
- コード例はすべて授業用に書き直したオリジナル版です。実際のプロジェクトのコードとは変数名などが異なります



参考 eBook： Unity3D.jp

<https://unity3d.jp/game/game-ebooks/level-up-your-code-with-game-programming-patterns/>



サンプルプロジェクト： GitHub

<https://github.com/Unity-Technologies/game-programming-patterns-demo/>

LESSON

ステートパターン

目標：状態管理を整理し、マリオ風キャラクターを実際に作れるようになる



このコマの目標

- if 文・ switch 文で状態を管理することの問題点を体感する
- ステートパターンの考え方（Enter・Update・Exit）を理解する
- IState インターフェイスと StateMachine クラスを実装できるようになる
- Idle・Walk・Jump の基本状態から、機能を 1 つずつ追加してマリオ風キャラクターを作る

悪い例： switch 文で状態を管理する

c# Program.cs

```
1 public enum CharacterState
2 {
3     Idle,
4     Walk,
5     Jump
6 }
7 public class NaiveCharacter : MonoBehaviour
8 {
9     private CharacterState state;
10    void Update()
11    {
12        switch (state)
13        {
14            case CharacterState.Idle:
15                UpdateIdle();
16                break;
17            case CharacterState.Walk:
18                UpdateWalk();
19                break;
20            case CharacterState.Jump:
21                UpdateJump();
22                break;
23        }
24    }
25 }
```

悪い例の問題点

- 状態が増えるほど、switch 文がどんどん長くなっていく
- 1 つの Update メソッドに、すべての状態の処理が詰め込まれてしまう
- 新しい状態（例：しゃがむ、攻撃する）を追加するたびに、この巨大な switch 文を修正する必要がある
- 「どこからどこへ状態が移れるか」というルールが、コードのあちこちに散らばりやすい

超悪い例：すべて if 文でチェックする

c# Program.cs

```
1 public class SuperNaiveMario : MonoBehaviour
2 {
3     private bool isIdle = true;
4     private bool isRunning;
5     private bool isJumping;
6
7     void Update()
8     {
9         if (isIdle && !isRunning && !isJumping) UpdateIdle();
10        if (isRunning && !isJumping) UpdateRun();
11        if (isJumping) UpdateJump();
12
13        if (!isJumping && Input.GetButtonDown("Jump"))
14        {
15            isJumping = true;
16            // isIdle と isRunning を false にし忘れると...?
17        }
18    }
19 }
```

状態遷移の図解



矢印は「状態遷移」を表す。同時にアクティブな状態は必ず 1 つだけ。

- 移動していなければ Idle、動けば Walk、ジャンプボタンを押せば Jump へ遷移する、といったイメージ。

ステートパターンとは

- 「状態」を1つずつ独立したクラスとして表現し、状態ごとの処理をそのクラスの中に閉じ込めるパターン
- 現在の状態オブジェクトを1つ持っておき、状態が変わったら参照先を差し替えるだけでよい
- 各状態は Enter（入った瞬間）・Update（毎フレーム）・Exit（出る瞬間）の3つの区切りで処理を持つ
 - － switch 文1つに全部書く代わりに、状態の数だけ小さなクラスに分ける、というイメージ

IState インターフェース

- すべての状態クラスが実装する共通のインターフェース。Enter ・ Update ・ Exit の 3 つを持つ。

C# Program.cs

```
1 public interface IState
2 {
3     void Enter();
4     void Update();
5     void Exit();
6 }
```

POINT

Enter = 状態に入った瞬間、Update = その状態にいる間ずっと、Exit = 状態から出る瞬間、に呼ばれる。

状態クラスの例: IdleState

c# Program.cs

```
1 public class IdleState : IState
2 {
3     private Character character;
4     public IdleState(Character character)
5     {
6         this.character = character;
7     }
8     public void Enter()
9     {
10         // 待機状態に入った直後の処理
11     }
12     public void Update()
13     {
14         if (character.IsMoving)
15         {
16             character.StateMachine.TransitionTo(
17                 character.StateMachine.WalkState);
18         }
19     }
20     public void Exit()
21     {
22         // 待機状態から抜ける直前の処理
23     }
24 }
```

StateMachine クラス

- 現在の状態を保持し、状態の切り替え（遷移）をまとめて管理するクラス。

c# Program.cs

```
1 public class StateMachine
2 {
3     public IState CurrentState { get; private set; }
4     public void Initialize(IState startState)
5     {
6         CurrentState = startState;
7         CurrentState.Enter();
8     }
9     public void TransitionTo(IState nextState)
10    {
11        CurrentState.Exit();
12        CurrentState = nextState;
13        CurrentState.Enter();
14    }
15    public void Update()
16    {
17        CurrentState.Update();
18    }
19 }
```

状態を切り替えて使ってみる

c# Program.cs

```
1  Character character = new Character();  
2  
3  // 最初の状態を設定  
4  character.StateMachine.Initialize(character.StateMachine.IdleState);  
5  
6  // 毎フレーム呼び出す  
7  character.StateMachine.Update();  
8  
9  // IdleState の Update 内で条件を満たすと  
10 // 自動的に WalkState へ TransitionTo される
```

Unity で作ってみよう： マリオ風キャラクター

- お題： Idle（待機）・Walk（歩く）・Jump（ジャンプ）を切り替えるマリオ風キャラクター
- 左右キーで移動、ジャンプキーでジャンプする、横スクロールアクションの基本操作
- Rigidbody2D を使って、移動とジャンプの物理挙動を扱う
- ここまで学んだ IState / StateMachine の仕組みを、そのままキャラクターに応用する

MarioCharacter クラス①: フィールド

c# Program.cs

```
1 using UnityEngine;
2 public class MarioCharacter : MonoBehaviour
3 {
4     public StateMachine stateMachine { get; private set; }
5     public IdleState idleState;
6     public WalkState walkState;
7     public JumpState jumpState;
8
9     public float moveSpeed = 5f;
10    public float jumpPower = 8f;
11
12    [SerializeField] float rayRange = 0.8f;
13    public bool isGrounded;
14
15    [Header("下記 GetComponent 記載済み")]
16    public Rigidbody2D rb;
17    public Animator anim;
18    public SpriteRenderer sr;
```

MarioCharacter クラス②: Awake

- コンポーネントの取得と、各 State のインスタンス化をまとめて行う。

C# Program.cs

```
1 void Awake()  
2 {  
3     rb = GetComponent<Rigidbody2D>();  
4     anim = GetComponent<Animator>();  
5     sr = GetComponent<SpriteRenderer>();  
6  
7     stateMachine = new StateMachine();  
8     idleState = new IdleState(this);  
9     walkState = new WalkState(this);  
10    jumpState = new JumpState(this);  
11 }
```

MarioCharacter クラス③: Start ・ Update

c# Program.cs

```
1 void Start()
2 {
3     // 最初は Idle 状態からスタート
4     stateMachine.Initialize(idleState);
5
6     // カメラ追従設定
7     CameraFollow.Instance.SetTarget(transform);
8 }
9
10 void Update()
11 {
12     // 着地判定の更新
13     CheckGround();
14
15     // 現在の状態の Update を呼ぶだけでよい
16     stateMachine.Update();
17 }
```

MarioCharacter クラス④：接地判定

- 足元に Ray を飛ばして、地面に触れているかどうかを毎フレーム判定する。

C# Program.cs

```
1 private void CheckGround()
2 {
3     int layerMask = ~(1 << LayerMask.NameToLayer("Player"));
4     RaycastHit2D hit = Physics2D.Raycast(
5         transform.position, Vector2.down, rayRange, layerMask);
6
7     if (hit.collider != null)
8     {
9         isGrounded = true;
10    }
11    else
12    {
13        isGrounded = false;
14    }
15 }
16
17 private void OnDrawGizmosSelected()
18 {
19     // 着地用のレイの可視化
20     Debug.DrawRay(transform.position, Vector2.down * rayRange, Color.red);
21 }
```

IdleState の実装（マリオ版）

c#

IdleState.cs

```
1 public class IdleState : IState
2 {
3     private MarioCharacter mario;
4     public IdleState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.anim.Play("Idle-Animation");
11     }
12     public void Update()
13     {
14         float input = Input.GetAxisRaw("Horizontal");
15         if (Mathf.Abs(input) > 0.1f)
16         {
17             mario.stateMachine.TransitionTo(mario.walkState);
18         }
19         else if (Input.GetButtonDown("Jump") && mario.isGrounded)
20         {
21             mario.stateMachine.TransitionTo(mario.jumpState);
22         }
23     }
24     public void Exit() { }
25 }
```

WalkState の実装（マリオ版）

c# WalkState.cs

```
1 public void Enter()
2 {
3     mario.anim.Play("run-Animation");
4 }
5 public void Update()
6 {
7     float input = Input.GetAxisRaw("Horizontal");
8     mario.rb.linearVelocity = new Vector2(
9         input * mario.moveSpeed, mario.rb.linearVelocity.y);
10
11     if (Mathf.Abs(input) < 0.1f)
12     {
13         mario.stateMachine.TransitionTo(mario.idleState);
14     }
15     else if (Input.GetButtonDown("Jump") && mario.isGrounded)
16     {
17         mario.stateMachine.TransitionTo(mario.jumpState);
18     }
19 }
```

WalkState の実装：向きの反転

- 入力の正負にあわせて `SpriteRenderer.flipX` を切り替え、キャラクターの向きを反転させる。

C# WalkState.cs (続き)

```
1 // WalkState.Update の続き
2 if (input > 0)
3 {
4     mario.sr.flipX = false;
5 }
6 else if (input < 0)
7 {
8     mario.sr.flipX = true;
9 }
```

JumpState の実装（マリオ版）

c# JumpState.cs

```
1 using UnityEngine;
2 public class JumpState : IState
3 {
4     private MarioCharacter mario;
5     public JumpState(MarioCharacter mario)
6     {
7         this.mario = mario;
8     }
9     public void Enter()
10    {
11        // 入った瞬間に上向き速度を与える
12        mario.rb.linearVelocity = new Vector2(
13            mario.rb.linearVelocity.x, mario.jumpPower);
14        mario.anim.Play("jump-Animation");
15    }
16    public void Update()
17    {
18        if (mario.isGrounded)
19        {
20            mario.stateMachine.TransitionTo(mario.idleState);
21        }
22    }
23    public void Exit() { }
24 }
```

Unity 上での設定手順

- ① キャラクター用の GameObject に、Rigidbody2D ・ Collider2D ・ Animator ・ SpriteRenderer を用意する
- ② Animator に「Idle-Animation」「run-Animation」「jump-Animation」のアニメーションを登録する
- ③ MarioCharacter.cs をアタッチし、IdleState / WalkState / JumpState のスクリプトを用意する
- ④ Project Settings > Input Manager に「Horizontal」「Jump」が設定されていることを確認する
- ⑤ 実行して、Idle → Walk → Jump と状態が切り替わることを確認する

POINT

接地判定は float rayRange の長さを足元に飛ばす Ray で行っている。地面のレイヤー設定に注意。

発展：機能を1つずつ追加していく

- 基本の Idle ・ Walk ・ Jump ができたら、状態を1つずつ追加して手触りを良くしていきます。
- ① 落下中（Fall）：ジャンプの上昇と下降を分ける
- ② コヨーテタイム：地面を離れた直後の一瞬もジャンプできるようにする
- ③ ジャンプ力の調整：ボタンを押す長さでジャンプの高さを変える
- ④ ヒップドロップ：空中で下入力をする、勢いよく地面へ降下する

POINT

ポイントは、既存の状態やクラスをできるだけ壊さずに「追加」していくことです。

① 落下中（ FallState ） の追加

C# FallState.cs

```
1 public class FallState : IState
2 {
3     private MarioCharacter mario;
4     public FallState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter() { }
9     public void Update()
10    {
11        if (mario.isGrounded)
12        {
13            mario.stateMachine.TransitionTo(mario.idleState);
14        }
15    }
16    public void Exit() { }
17 }
```

C# JumpState.cs (追記)

```
1 if (mario.rb.linearVelocity.y < 0)
2 {
3     mario.stateMachine.TransitionTo(mario.fallState);
4 }
```

② コヨーテタイムの追加

C# MarioCharacter.cs
(追記)

```
1 public float coyoteTime = 0.15f;  
2 public float coyoteTimeCounter;
```

C# FallState.cs (Update 書き換え)

```
1 public void Update()  
2 {  
3     mario.coyoteTimeCounter -= Time.deltaTime;  
4     if (mario.coyoteTimeCounter > 0 && Input.GetButtonDown("Jump"))  
5     {  
6         mario.stateMachine.TransitionTo(mario.jumpState);  
7     }  
8     if (mario.isGrounded)  
9     {  
10        mario.stateMachine.TransitionTo(mario.idleState);  
11    }  
12 }
```

C# IdleState/
WalkState.cs (Exit 追記)

```
1 public void Exit()  
2 {  
3     mario.coyoteTimeCounter = mario.coyoteTime;  
4 }
```

③ ジャンプ力の調整（ボタン長押し）

- ジャンプボタンを離した瞬間に上向きを速度を弱める。短く押すと低いジャンプ、長く押すと高いジャンプになる。

C# JumpState.cs (追記)

```
1 public void Update()
2 {
3     if (Input.GetButtonUp("Jump") && mario.rb.linearVelocity.y > 0)
4     {
5         mario.rb.linearVelocity = new Vector2(
6             mario.rb.linearVelocity.x, mario.rb.linearVelocity.y * 0.5f);
7     }
8     if (mario.rb.linearVelocity.y < 0)
9     {
10         mario.stateMachine.TransitionTo(mario.fallState);
11     }
12 }
```

④ ヒップドロップ①: HipDropState の追加

C# MarioCharacter.cs
(追記)

```
1 public float hipDropSpeed = 12f;
```

C# HipDropState.cs

```
1 public class HipDropState : IState
2 {
3     private MarioCharacter mario;
4     public HipDropState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.rb.linearVelocity = new Vector2(0f, -mario.hipDropSpeed);
11         mario.anim.Play("hipdrop-Animation");
12     }
13     public void Update()
14     {
15         if (mario.isGrounded)
16         {
17             mario.stateMachine.TransitionTo(mario.idleState);
18         }
19     }
20     public void Exit() { }
21 }
```

④ ヒップドロップ②：発動条件の追加

- 空中（Jump・Fall中）に下入力をする、ヒップドロップへ遷移する。

```
JumpState.cs /  
c# FallState.cs (追記)  
1  if (Input.GetAxisRaw("Vertical") < -0.5f)  
2  {  
3      mario.stateMachine.TransitionTo(mario.hipDropState);  
4  }
```

POINT 地面に着いたら HipDropState の Update で Idle へ戻る。着地時のエフェクトを足すと迫力が出ます。

完成した状態遷移の図解

Walk

Idle

Jump

HipDrop

Fall

- Idle/Walk ⇄ Jump ⇄ Fall ⇄ HipDrop という形で、状態が少しずつつながっていく。
- コヨーテタイムやジャンプ力の調整は、既存の状態の中の「処理」として追加されている点に注目。

最後に：もしステートパターンを使わなかったら

- ここまで作ってきた Idle ・ Walk ・ Jump ・ Fall ・ コヨーテタイム ・ 可変ジャンプ ・ ヒップドロップ
- これを、状態クラスに分けず、関数にもまとめず、Update の中にひたすら書いていくとどうなるでしょうか？
- 次のページで実際に見てみましょう。

注意

IState も StateMachine も使わず、すべて bool 変数と if 文だけで済ませた場合の例です。

超・悪い例：全部 Update に書く①

C# MarioCharacter.cs
(悪い例)

```
1 void Update()
2 {
3     RaycastHit2D hit = Physics2D.Raycast(
4         transform.position, Vector2.down, rayRange);
5     isGrounded = hit.collider != null;
6
7     float input = Input.GetAxisRaw("Horizontal");
8     float vertical = Input.GetAxisRaw("Vertical");
9
10    if (isGrounded)
11    {
12        coyoteTimeCounter = coyoteTime;
13    }
14    else
15    {
16        coyoteTimeCounter -= Time.deltaTime;
17    }
```

超・悪い例：全部 Update に書く②

MarioCharacter.cs

C#
(悪い例・続き)

```
1      if (isHipDropping)
2      {
3          if (isGrounded)
4          {
5              isHipDropping = false;
6              isIdle = true;
7              anim.Play("Idle-Animation");
8          }
9      }
10     else if (vertical < -0.5f && !isGrounded)
11     {
12         isHipDropping = true;
13         isJumping = false;
14         isFalling = false;
15         rb.linearVelocity = new Vector2(0f, -hipDropSpeed);
16         anim.Play("hipdrop-Animation");
17     }
```

超・悪い例：全部 Update に書く③

MarioCharacter.cs

C# (悪い例・続き)

```
1  else if (isJumping)
2  {
3      if (Input.GetButtonUp("Jump") && rb.linearVelocity.y > 0)
4      {
5          rb.linearVelocity = new Vector2(
6              rb.linearVelocity.x, rb.linearVelocity.y * 0.5f);
7      }
8      if (rb.linearVelocity.y < 0)
9      {
10         isJumping = false;
11         isFalling = true;
12     }
13 }
14 else if (isFalling)
15 {
16     if (isGrounded)
17     {
18         isFalling = false;
19         isIdle = true;
20     }
21 }
```

超・悪い例：全部 Update に書く④

MarioCharacter.cs

C#
(悪い例・続き)

```
1     else if (isWalking || isIdle)
2     {
3         rb.linearVelocity = new Vector2(
4             input * moveSpeed, rb.linearVelocity.y);
5
6         if (Mathf.Abs(input) > 0.1f)
7         {
8             isWalking = true; isIdle = false;
9             anim.Play("run-Animation");
10        }
11        else
12        {
13            isIdle = true; isWalking = false;
14            anim.Play("Idle-Animation");
15        }
16
17        if (input > 0) sr.flipX = false;
18        else if (input < 0) sr.flipX = true;
```

超・悪い例：全部 Update に書く ⑤

MarioCharacter.cs

C#
(悪い例・続き)

```
1      if (Input.GetButtonDown("Jump") &&
2          (isGrounded || coyoteTimeCounter > 0))
3      {
4          isJumping = true; isWalking = false; isIdle = false;
5          rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpPower);
6          anim.Play("jump-Animation");
7      }
8  }
9  }
```

超・悪い例から見えること

- `isIdle / isWalking / isJumping / isFalling / isHipDropping ... bool` 変数が状態の数だけ増えていく
- `if / else if` の 1 つの塊がどんどん長くなり、条件の組み合わせを人間が頭の中で追わないといけな
- 新しい状態（二段ジャンプ、しゃがみなど）を足すたびに、この巨大な Update をまた書き換えることになる
- ステートパターンは、この「1 つの塊」を状態ごとに小さく切り分けるための仕組みだった、というのが今日のまとめです。

長所と短所

長所

- 状態ごとの処理が独立したクラスに分かれ、見通しが良くなる
- 新しい状態を追加しても、既存の状態にほとんど影響しない
- 「どの状態からどの状態へ移れるか」を状態クラスの中に閉じ込められる

短所

- 状態の数だけクラスが増え、ファイル数が多くなる
- 状態が少ない・変化がほぼ無い場合は、switch 文の方がシンプルなこともある
- 状態同士のやり取りが複雑になると、遷移のルールを追うのが大変になる

このコマのまとめ

- **ステートパターン=状態ごとの処理を、独立したクラスに分けて管理するパターン**
- IState（Enter・Update・Exit）を実装し、StateMachineで現在の状態を管理する
- if文・switch文が肥大化しやすい場面（状態が多い・今後が増える）で特に効果を発揮する
- Idle・Walk・Jumpの基本形から、Fall・コヨーテタイム・可変ジャンプ・ヒップドロップを1つずつ追加した
- 既存の状態やクラスをできるだけ壊さずに機能を積み重ねられるのも、ステートパターンの強み

お疲れさまでした

今日作ったキャラクターに、しゃがむ・攻撃するなどの状態を追加してみると、パターンの効果をさらに実感できます。